```
$ cat title.txt
Contest Codebook

$ cat institution.txt
Mymensingh Engineering College

$ whoami
MEC_ALGO_SCHOLARS

$
```

TEMPLATE — PART 1/3

```cpp
#include <bits/stdc++.h>

using namespace std;

#define loop(a,b,c)      for(int a=b;a<c;a++)
#define loopr(a,b,c)     for(int a=b;a>c;a--)
#define nl               ("\n")
#define int              long long
#define double           long double
#define vi               vector<int>
#define pi               pair<int, int>
#define vb               vector<bool>
#define vs               vector<string>
#define pb(n)            push_back(n)
#define eb(n)            emplace_back(n)
#define bitcntll(n)      __builtin_popcountll(n)
#define ff               first
#define ss               second
#define sz(s)            (long long)(s.size())
#define all(v)           (v).begin(),(v).end()
#define sp(n)            cout<<setprecision(n)<<fixed;
#define in(v)            for(auto &item : v) cin>>item;
#define inp(v)           for(auto &item : v) cin>>item.ff>>item.ss;
#define google(T)        cout<< "Case #" << T << ": ";
#define  yn(f)           cout<<(f? "YES\n":"NO\n");


#ifndef ONLINE_JUDGE
#define db(x)            cerr << #x <<" "; _print(x); cerr << nl;
#define gt(T)            cerr << "Case #" << T << ": " << nl;
#else
#define db(x)
#define gt(T)
#endif

void _print(int t) {cerr << t;}
void _print(double t) {cerr << t;}
void _print(string t) {cerr << t;}
void _print(char t) {cerr << t;}
void _print(bool t) {cerr << ((t)?"true":"false");}


template <class T, class V> void _print(pair <T, V> p) {cerr << "{"; _print(p.ff); cerr << ","; _print(p.ss);
cerr << "}";}
template <class T> void _print(vector <T> v) {cerr << "[ "; for (T i : v) {_print(i); cerr << " ";} cerr << "]";}
template <class T> void _print(set <T> v) {cerr << "[ "; for (T i : v) {_print(i); cerr << " ";} cerr << "]";}
template <class T> void _print(multiset <T> v) {cerr << "[ "; for (T i : v) {_print(i); cerr << " ";} cerr <<
"]";}
```

TEMPLATE — PART 2/3

```cpp
template <class T, class V> void _print(map <T, V> v) {cerr << "[ "; for (auto i : v) {_print(i); cerr << " ";}
cerr << "]";}
template <class T, class V> void _print(unordered_map <T, V> v) {cerr << "[ "; for (auto i : v) {_print(i); cerr
<< " ";} cerr << "]";}


inline void fastio()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
}


// int dx[] = { -1 , 1 ,  0  , 0 , -1  , -1  ,  1  , 1  };
// int dy[] = { 0  , 0 , -1  , 1 , -1  ,  1  , -1  , 1  };

//            { U  , D ,  L  , R , UL  , UR  , DL  , DR }









/*-------------------------------------------------------------------------------------------------------------
------------*/
int modadd(int a,int b,int m){ a %= m;b %= m; return (a + b) % m;}
int modmul(int a,int b,int m){ a %= m;b %= m; return (a * b) % m;}
int modsub(int a,int b,int m){ a %= m;b %= m; return (a - b + m) % m;}
int gcd(int a, int b){ if(b == 0) return a; return gcd(b, a % b);}
int lcm(int a, int b){ return a*b/gcd(a, b);}
int expo(int a,int n,int md){ int res=1; while(n){ if(n&1) {res = modmul(res,a,md);--n;} else {a =
modmul(a,a,md);n >>= 1;}} return res;}
int expo(int a,int n){ int res=1; while(n){ if(n&1) {res *= a;--n;} else {a *= a;n >>= 1;}} return res;}
template <typename T> bool revsort(T a, T b){return a > b;}
vb sieve(int n){vb prime(n+1,1); prime[0] = prime[1] = 0; for(int i=2;i*i<=n;i++){if(prime[i]){for(int
j=i*i;j<=n;j+=i) prime[j]=0;}} return prime;}
vs rmspace(string s){ vs toreturn; string tmp = ""; for(auto i : s){ if(i == ' '){ toreturn.pb(tmp); tmp = "";}
else tmp += i;} toreturn.pb(tmp); return toreturn;}
int modInv(int a, int m){if (__gcd(a, m) != 1)return -1; else return expo(a, m - 2, m);}
/*-------------------------------------------------------------------------------------------------------------
------------*/

const double PI = 3.1415926535;
const int inf = 1e18;
const int mod = 1000000007;

void solve(int T)
{
}


signed main()
```

TEMPLATE — PART 3/3

```
{
    fastio();
    int T = 1;
    cin >> T;
    for(int i = 1; i <= T; i++)
        solve(i);
    return 0;
}
```

BINARY SEARCH

SquareRootBS

O(log n)

```
int main(){
    vector<int>v;
    int searchVal;
    ll lo = 0, hi = v.size()-1, mid;
    ll result = 0;
    while(lo<=hi){
        mid = (lo+hi)/2;
        if(v[mid]<=searchVal){
            lo = mid+1;
            result = mid;
        }else{
            hi = mid-1;
        }
    }
    cout<<result<<endl;
}
```

BIT MANIPULATION

Subset

```
vector<int>v(n);
for(int i=0;i<n;i++){
    cin>>v[i];
}
int subset_ct = (1<<n);
for(int i=0;i<subset_ct;i++){
    for(int j=0;j<n;j++){
        if((i&(1<<j))!=0){
            cout<<v[j]<<' ';
        }
    }
    cout<<endl;
}
```

NUMBER THEORY

Sieve Of Erathones

O(nlog(log(n)))

Union by size, path compression

```
const int N = 1e7 +10;
vector<bool>isPrime(N, 1);
int main(){
    isPrime[0] = isPrime[1] = false;
    for(int i=2;i<N;i++){
        if(isPrime[i]){
            for(int j = 2*i;j<N;j+=i){
                isPrime[j] = false;
            }
        }
    }
    int cnt = 0;
    for(int i=1;i<100;i++){
        cout<<isPrime[i]<<endl;
        if(isPrime[i])cnt++;
    }
```

Sieve Of Erathones (cont.)

O(nlog(log(n)))

```
    cout<<cnt<<endl;
}
```

Linear Sieve

O(n)

```
const int N = 1e7 + 5;
vector<int> primes;
vector<int> spf(N, 0);   // smallest prime factor

void linear_sieve(int n){
    for(int i = 2; i <= n; i++){
        // i is prime
        if(spf[i] == 0){
            spf[i] = i;
            primes.push_back(i);
        }
        for(int p : primes){
            if(1LL * i * p > n) break;
            // mark i*p
            spf[i * p] = p;
            // stop when p becomes the smallest
prime factor of i
            if(p == spf[i]) break;
        }
    }
}
```

Smallest Prime Factor

```
const int N = 1e7 + 5;
vector<int> spf(N);
void sieve(){
    for(int i = 0; i < N; i++)
        spf[i] = i;
    for(int i = 2; i * i < N; i++){
        if(spf[i] == i){
            for(int j = i * i; j < N; j += i){
                if(spf[j] == j)
                    spf[j] = i;
            }
        }
    }
}
vector<int> primeFactor(int x){
    vector<int> factors;
    while(x > 1){
        factors.push_back(spf[x]);
        x /= spf[x];
    }
    return factors;
}
```

Inverse Modulo

O(log n)

```
long long modInverse(long long a, long long mod){

    return power(a, mod - 2, mod);
}
```

Euler Phi Sieve

```
const int N = 1000000;

vector<int> phi(N + 1);

void phiSieve(){

    for(int i = 0; i ⩽ N; i++)
        phi[i] = i;

    for(int i = 2; i ⩽ N; i++){

        if(phi[i] == i){ // i is prime

            for(int j = i; j ⩽ N; j += i)
                phi[j] -= phi[j] / i;
        }
    }
}
```

COMBINATORICS

Catalan Numbers

```
for(int i = 1; i < MAXN; i++){
    for(int j = 0; j < i; j++){
        catalan[i] = (catalan[i] +
                      catalan[j] * catalan[i-1-j])
% MOD;
    }
}
```

Factorial

```
typedef long long ll;
const int MAXN = 1e6;
const int MOD = 1e9+7;
ll fac[MAXN];

void compute_factorials(){
    fac[0] = 1LL;
    for(int i=1;i<MAXN;i++){
        fac[i] = (fac[i-1]*i)%MOD;
    }
}
```

nCr

```
typedef long long ll;
const int MAXN = 1e6;
const int MOD = 1e9+7;
ll fac[MAXN], infac[MAXN];

ll expo(ll a, ll b){
    if(b==0){
        return 1;
    }
    if(b%2==0){
        ll x = expo(a, b/2);
        return (x*x)%MOD;
```

nCr (cont.)

```
    }else{
        ll x = expo(a, b/2);
        ll ans = (x*x)%MOD;
        return (ans*a)%MOD;
    }
}

ll inverse(ll a){
    return expo(a, MOD-2);
}

void compute_factorials(){
    fac[0] = 1;
    infac[0] = 1;
    for(int i=1;i<MAXN;i++){
        fac[i] = (fac[i-1]*i)%MOD;
        infac[i] = inverse(fac[i]);
    }
}

// nCr = n! / r! * (n - r)!
ll nCr(ll n, ll r){
    if(n<0 or r<0 or r>n){
        return 0;
    }
    ll num = fac[n];
    ll den = (infac[r])*(infac[n-r])%MOD;
    return (num*den)%MOD;
}

int main(){
    compute_factorials();
    int n, m;cin>>n>>m;
    cout<<nCr(m, n)<<endl;
}
```

GRAPH

DSU

```
vector<int> prefix_fn(string& s){
  int n=s.size();
  vector<int> pi(n);
  for(int i=1;i<n;i++){
    int j=pi[i-1];
    while(j>0 && s[i]≠s[j]) j=pi[j-1];
    if(s[i]==s[j]) j++;
    pi[i]=j;
  }
  return pi;
}
```

BFS

```
const int N = 1e5;
vector<int> graph[N];
int vis[N];
int level[N];

void bfs(int source){
```

BFS (cont.)

```
    queue<int>q;
    q.push(source);
    vis[source] = 1;

    while(!q.empty()){
        int cur_v = q.front();
        q.pop();
        cout<<cur_v<<' ';
        for(auto child: graph[cur_v]){
            if(!vis[child]){
                q.push(child);
                vis[child] = 1;
                level[child] = level[cur_v] + 1;
            }
        }
    }
}
```

DFS

```
void dfs(int vertex){
    // Take action on vertex after entering the
vertex
    if(vis[vertex])return ;
    vis[vertex] = true;
    cout<<vertex<<endl;
    for(auto child:graph[vertex]){
        // Take action on a child before entering
the child node
        cout<<"Parent "<<vertex<<", child "
<<child<<endl;
        dfs(child);
        // Take action on a child after existing
child node
    }
    // Take action on vertex before exiting the
vertex
}
```

Bi Coloring

```
#define N 200010

vector<int>adj[N];
int visited[N];
int color[N];
int is_possible = 1;
void dfs(int u, int c){
    if(visited[u]){
        if(color[u]==c){
            return ;
        }else{
            is_possible = 0;
            return ;
        }
    }
    visited[u] = 1;
    color[u] = c;
    for(int v:adj[u]){
        dfs(v, 3-c);
    }
}

int main(){
    int n, m;
    cin>>n>>m;
```

Bi Coloring (cont.)

```
    for(int i=0;i<m;i++){
        int u, v;
        cin>>u>>v;
        adj[v].push_back(u);
        adj[u].push_back(v);
    }
    for(int i=1;i<=n;i++){
        if(!visited[i])dfs(i, 1);
    }
    if(!is_possible){
        cout<<"IMPOSSIBLE"<<endl;
    }else{
        for(int i=1;i<=n;i++){
            cout<<color[i]<<' ';
        }
        cout<<endl;
    }
}
```

Cycle Detection

```
int n, m;
vector<int>adj[N];
int previous_node[N];
int visited[N];
bool cycle_found = false;
int cycle_start, cycle_end;

void dfs(int u, int prev){
    if(visited[u]){
        if(cycle_found==false){
            cycle_found = true;
            cycle_start = u;
            cycle_end = prev;
        }
        return ;
    }
    visited[u] = 1;
    previous_node[u] = prev;
    for(int v:adj[u]){
        if(prev!=v)
            dfs(v, u);
    }
}


int main(){
    cin >> n >> m;

    for(int i = 0; i < m; i++){
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    for(int i = 1; i <= n; i++){
        if(!visited[i]) dfs(i, 0);
    }

    if(cycle_found){
        cout << "Cycle detected\n";
```

Cycle Detection (cont.)

```
        vector<int> path;
        int node = cycle_end;
        while(1){
            path.push_back(node);
            node = previous_node[node];
            if(node==cycle_start) break;
        }
        path.push_back(cycle_start);

        reverse(path.begin(), path.end());

        for(int node: path){
            cout << node << " → ";
        }
        cout << endl;
    }
}
```

Size of Subtree

```
#define N 100000

vector<int>adj[N];
int size[N];
void dfs(int u, int par){
    size[u] = 1;
    for(int v:adj[u]){
        if(v==par)continue;
        dfs(v, u);
        size[u] += size[v];
    }
}
```

Topological Order DFS

```
#define N 2000005
vector<int>adj[N];
int visited[N];
vector<int>topological_order;

void dfs(int u){
    visited[u] = 1;
    for(int v:adj[u]){
        if(visited[v])continue;
        dfs(v);
    }
    topological_order.push_back(u);
}
int main(){
    int n, m;
    cin >> n >> m;
    for(int i=0;i<m;i++){
        int u, v;
        cin>>u>>v;
        adj[u].push_back(v);
    }
    for(int i=1;i<=n;i++){
        if(!visited[i])dfs(i);
    }
    reverse(topological_order.begin(),
topological_order.end());
    for(int node:topological_order){
```

Topological Order DFS (cont.)

```
        cout<<node<<' ';
    }
}
```

Xor On Tree

```
#define N 100000

vector<pair<int, int>>adj[N];
int preXor[N];

void dfs(int u, int par){
    for(auto child:adj[u]){
        int v = child.first;
        int w = child.second;
        if(v==par)continue;
        preXor[v] = preXor[u]^w;
        dfs(v, u);
    }
}

int main(){
    int n;
    cin>>n;
    for(int i=0;i<n-1;i++){
        int u, v, w;
        cin>>u>>v>>w;
        adj[v].push_back({u, w});
        adj[u].push_back({v, w});
    }
    dfs(1, -1);
    // xor between 3 and 5;
    cout<<(preXor[1] ^ preXor[4])<<endl;
}
```

Subtree Sum

```
const int N = 1e5;
vector<int> graph[N];
int subtree_sum[N], even_ct[N];

void dfs(int vertex, int parent){
    // Take action on vertex after entering the
vertex
    if(vertex%2==0){
        even_ct[vertex]++;
    }
    subtree_sum[vertex] += vertex;
    for(auto child:graph[vertex]){
        // Take action on a child before entering
the child node
        if(child==parent)continue;
        dfs(child, vertex);
        subtree_sum[vertex] += subtree_sum[child];
        even_ct[vertex] += even_ct[child];
        // Take action on a child after existing
child node
    }
    // Take action on vertex before exiting the
vertex
}

int main(){
    int n;
    cin>>n;
```

Subtree Sum (cont.)

```
    for(int i=0;i<n-1;i++){
        int v1, v2;cin>>v1>>v2;
        graph[v1].push_back(v2);
        graph[v2].push_back(v1);
    }
    dfs(1, -1);
    for(int i=1;i<=n;i++){
        cout<<subtree_sum[i]<<' '<<even_ct[i]
<<endl;
    }
}
```

Number of Islands

```
 void dfs(int i,int j, vector<vector<char>>& grid,
vector<vector<int>>& vis){
        int n = grid.size();
        int m = grid[0].size();
        if(i<0 or j<0 or i>=n or j>=m or grid[i]
[j]=='0' or vis[i][j])return ;
        vis[i][j] = 1;

        dfs(i-1, j, grid, vis);
        dfs(i+1, j, grid, vis);
        dfs(i, j+1, grid, vis);
        dfs(i, j-1, grid, vis);
    }
    int numIslands(vector<vector<char>>& grid) {
        int row = grid.size();
        int col = grid[0].size();
        int ans = 0;
        vector<vector<int>> vis(row, vector<int>
(col, 0));
        for(int i=0;i<row;i++){
            for(int j=0;j<col;j++){
                if(vis[i][j]==0 and grid[i]
[j]=='1'){
                    dfs(i, j, grid, vis);
                    ans++;
                }
            }
        }
        return ans;
    }

int main(){
    int n, m;cin>>n>>m;
    vector<vector<char>>vv;
    for(int i=0;i<n;i++){
        vector<char>v;
        for(int j=0;j<m;j++){
            char x;cin>>x;
            v.push_back(x);
        }
        vv.push_back(v);
    }
    cout<<numIslands(vv)<<endl;

}
```

Prims Minimum Spanning Tree

```
const int N = 1e5 + 5;

vector<pair<int,int>> adj[N];
```

Prims Minimum Spanning Tree (cont.)

```
void prim(int source, int n){

    vector<int> vis(n + 1, 0);

    // {weight, node}
    priority_queue<pair<int,int>,
                   vector<pair<int,int>>,
                   greater<pair<int,int>>> pq;

    pq.push({0, source});

    int mst_cost = 0;

    while(!pq.empty()){

        auto [wt, u] = pq.top();
        pq.pop();

        if(vis[u]) continue;

        vis[u] = 1;
        mst_cost += wt;

        for(auto [v, w] : adj[u]){

            if(!vis[v]){
                pq.push({w, v});
            }
        }
    }

    cout << "MST Cost = " << mst_cost << '\n';
}
```

Kruskal + DSU MST

```
#include <bits/stdc++.h>
using namespace std;

#define int long long

struct Edge{
    int u, v, w;

    bool operator<(const Edge& other) const{
        return w < other.w;
    }
};

const int N = 2e5 + 5;

int parent[N], sz[N];

void make(int v){
    parent[v] = v;
    sz[v] = 1;
}

int find(int v){
    if(v == parent[v]) return v;
    return parent[v] = find(parent[v]); // path
compression
}
```

Kruskal + DSU MST (cont.)

```
bool Union(int a, int b){
    a = find(a);
    b = find(b);

    if(a == b) return false; // cycle

    if(sz[a] < sz[b]) swap(a, b);

    parent[b] = a;
    sz[a] += sz[b];

    return true;
}

signed main(){
    ios::sync_with_stdio(0);
    cin.tie(0);

    int n, m;
    cin >> n >> m;

    vector<Edge> edges;

    for(int i = 0; i < m; i++){
        int u, v, w;
        cin >> u >> v >> w;
        edges.push_back({u, v, w});
    }

    for(int i = 1; i <= n; i++)
        make(i);

    sort(edges.begin(), edges.end());

    int mst_cost = 0;
    vector<Edge> mst_edges;

    for(auto e : edges){

        if(Union(e.u, e.v)){
            mst_cost += e.w;
            mst_edges.push_back(e);
        }
    }

    // If graph is disconnected
    if((int)mst_edges.size() != n - 1){
        cout << "MST does not exist\\n";
        return 0;
    }

    cout << mst_cost << '\\n';

    for(auto e : mst_edges){
        cout << e.u << " " << e.v << " " << e.w <<
'\\n';
    }
}
```

Dijkstra

O((V+E) log V)

Single source shortest path, non-negative weights

```
const int N = 1e5 + 5;
const int INF = 1e9;
```

Dijkstra (cont.)

O((V+E) log V)

```
vector<pair<int,int>> g[N];

void dijkstra(int source, int n){

    vector<int> dist(n + 1, INF);

    priority_queue<pair<int,int>,
vector<pair<int,int>>, greater<pair<int,int>>> pq;

    dist[source] = 0;
    pq.push({0, source});

    while(!pq.empty()){

        auto [d, v] = pq.top();
        pq.pop();

        // Ignore outdated entries
        if(d != dist[v]) continue;

        for(auto [child, wt] : g[v]){

            if(dist[v] + wt < dist[child]){

                dist[child] = dist[v] + wt;
                pq.push({dist[child], child});
            }
        }
    }

    // Print shortest distances
    for(int i = 1; i <= n; i++){
        cout << "Distance from " << source << " to
" << i << " = ";
        if(dist[i] == INF)
            cout << "INF\n";
        else
            cout << dist[i] << '\n';
    }
}
```

Bellman Ford

```
#include <bits/stdc++.h>
using namespace std;

#define int long long

const int INF = 1e18;

struct Edge{
    int u, v, w;
};

signed main(){

    int n, m;
    cin >> n >> m;

    vector<Edge> edges;

    for(int i = 0; i < m; i++){

        int u, v, w;
```

Bellman Ford (cont.)

```
        cin >> u >> v >> w;

        edges.push_back({u, v, w});
    }

    int source;
    cin >> source;

    vector<int> dist(n + 1, INF);
    dist[source] = 0;

    // Relax all edges (n-1) times
    for(int i = 1; i <= n - 1; i++){

        bool updated = false;

        for(auto e : edges){

            if(dist[e.u] == INF) continue;

            if(dist[e.u] + e.w < dist[e.v]){

                dist[e.v] = dist[e.u] + e.w;
                updated = true;
            }
        }

        // Optimization
        if(!updated) break;
    }

    // Detect negative cycle
    bool negativeCycle = false;

    for(auto e : edges){

        if(dist[e.u] == INF) continue;

        if(dist[e.u] + e.w < dist[e.v]){

            negativeCycle = true;
            break;
        }
    }

    if(negativeCycle){

        cout << "Negative Weight Cycle Exists\n";
        return 0;
    }

    for(int i = 1; i <= n; i++){

        if(dist[i] == INF)
            cout << "INF ";
        else
            cout << dist[i] << " ";
    }

    cout << '\n';
}
```

Floyd Warshall

```
#include <bits/stdc++.h>
using namespace std;

#define int long long

const int INF = 1e18;

signed main(){

    int n, m;
    cin >> n >> m;

    vector<vector<int>> dist(n + 1, vector<int>(n
+ 1, INF));

    // Distance from a node to itself
    for(int i = 1; i <= n; i++)
        dist[i][i] = 0;

    // Read edges
    for(int i = 0; i < m; i++){

        int u, v, w;
        cin >> u >> v >> w;

        dist[u][v] = min(dist[u][v], w);

        // Uncomment for undirected graph
        // dist[v][u] = min(dist[v][u], w);
    }

    // Floyd-Warshall
    for(int k = 1; k <= n; k++){

        for(int i = 1; i <= n; i++){

            if(dist[i][k] == INF) continue;

            for(int j = 1; j <= n; j++){

                if(dist[k][j] == INF) continue;

                dist[i][j] = min(dist[i][j],
                                 dist[i][k] +
dist[k][j]);
            }
        }
    }

    // Detect negative cycle
    for(int i = 1; i <= n; i++){

        if(dist[i][i] < 0){

            cout << "Negative Weight Cycle
Exists\n";
            return 0;
        }
    }

    // Print shortest distance matrix
    for(int i = 1; i <= n; i++){

        for(int j = 1; j <= n; j++){

            if(dist[i][j] == INF)
                cout << "INF ";
            else
                cout << dist[i][j] << " ";
```

Floyd Warshall (cont.)

```
        }

        cout << '\n';
    }
}
```

LCA with Binary Lifting

```
#include <bits/stdc++.h>
using namespace std;

const int N = 200005;
const int LOG = 20;

vector<int> graph[N];

int level[N];
int parent[N][LOG];

void dfs(int v, int par){

    parent[v][0] = par;

    for(int i = 1; i < LOG; i++){

        if(parent[v][i-1] == -1)
            parent[v][i] = -1;
        else
            parent[v][i] = parent[parent[v][i-1]]
[i-1];
    }

    for(auto child : graph[v]){

        if(child == par) continue;

        level[child] = level[v] + 1;

        dfs(child, v);
    }
}

int lca(int u, int v){

    if(level[u] < level[v])
        swap(u, v);

    // Bring u to the same level as v
    int diff = level[u] - level[v];

    for(int i = LOG - 1; i >= 0; i--){

        if(diff & (1 << i))
            u = parent[u][i];
    }

    if(u == v)
        return u;

    // Lift both together
    for(int i = LOG - 1; i >= 0; i--){

        if(parent[u][i] != parent[v][i]){
```

LCA with Binary Lifting (cont.)

```
            u = parent[u][i];
            v = parent[v][i];
        }
    }

    return parent[u][0];
}

int main(){

    int n;
    cin >> n;

    for(int i = 0; i < n - 1; i++){

        int u, v;
        cin >> u >> v;

        graph[u].push_back(v);
        graph[v].push_back(u);
    }

    memset(parent, -1, sizeof(parent));

    level[1] = 0;
    dfs(1, -1);

    int q;
    cin >> q;

    while(q--){

        int u, v;
        cin >> u >> v;

        cout << lca(u, v) << '\n';
    }
}
```

DYNAMIC PROGRAMMING

Flog 2

```
#include <bits/stdc++.h>
#define int long long
#define all(x) x.begin(), x.end()

using namespace std;

signed main(){
    int n, k;cin>>n>>k;
    vector<int>stones(n+1);
    for(int i=1;i<=n;i++){
        cin>>stones[i];
    }
    int inf = 1e9;
    vector<int>cost(n+1, inf);
    cost[1] = 0;
    for(int i=2;i<=n;i++){
        for(int j=1;j<=k;j++)
            if(i-j>=1)
                cost[i] = min(cost[i], cost[i-j] +
abs(stones[i]-stones[i-j]));
```

Flog 2 (cont.)

```
    }
    cout<<cost[n]<<endl;
}
```

LIS Binary Search

```
#include <bits/stdc++.h>
using namespace std;

#define int long long

int LIS(vector<int> &a){

    vector<int> tail;

    for(int x : a){

        auto it = lower_bound(tail.begin(),
tail.end(), x);

        if(it == tail.end())
            tail.push_back(x);

        else
            *it = x;
    }

    return tail.size();
}

signed main(){

    int n;
    cin >> n;

    vector<int> a(n);

    for(auto &x : a)
        cin >> x;

    cout << LIS(a) << '\n';
}
```

Knapsack

```
#include <bits/stdc++.h>
using namespace std;

int main(){

    int n, W;
    cin >> n >> W;

    vector<int> weight(n + 1);
    vector<int> value(n + 1);

    for(int i = 1; i <= n; i++)
        cin >> weight[i];

    for(int i = 1; i <= n; i++)
        cin >> value[i];
```

Knapsack (cont.)

```
    vector<vector<int>> dp(n + 1,
                           vector<int>(W + 1, 0));

    for(int i = 1; i <= n; i++){

        for(int w = 0; w <= W; w++){

            dp[i][w] = dp[i-1][w];

            if(weight[i] <= w){

                dp[i][w] = max(dp[i][w],
                               value[i] +
                               dp[i-1][w-
weight[i]]);
            }
        }
    }

    cout << dp[n][W] << '\n';
}
```

LCIS

```
#include <bits/stdc++.h>
using namespace std;

int main() {

    int n;
    cin >> n;

    vector<int> a(n + 1), b(n + 1);

    for(int i = 1; i <= n; i++) cin >> a[i];
    for(int i = 1; i <= n; i++) cin >> b[i];

    vector<int> dp(n + 1, 0);

    for(int i = 1; i <= n; i++) {

        int current = 0;

        for(int j = 1; j <= n; j++) {

            if(a[i] == b[j]) {
                dp[j] = max(dp[j], current + 1);
            }

            if(a[i] > b[j]) {
                current = max(current, dp[j]);
            }
        }
    }

    cout << *max_element(dp.begin(), dp.end()) <<
'\n';
}
```

LCS

```
#include <bits/stdc++.h>
using namespace std;
```

LCS (cont.)

```
int main(){

    string a, b;
    cin >> a >> b;

    int n = a.size();
    int m = b.size();

    vector<vector<int>> dp(n + 1,
                           vector<int>(m + 1, 0));

    for(int i = 1; i <= n; i++){

        for(int j = 1; j <= m; j++){

            if(a[i-1] == b[j-1])
                dp[i][j] = dp[i-1][j-1] + 1;

            else
                dp[i][j] = max(dp[i-1][j],
                               dp[i][j-1]);
        }
    }

    cout << dp[n][m] << '\n';
}
```

Two Set CSES

```
int n;
int dp[501][62626];
const int mod = 1e9 + 7;

int fun(int idx, int sum){
    if(sum * 2 > (n + 1) * n / 2) return 0;
    if(idx > n){
        if(sum * 2 == (n + 1) * n / 2){
            return 1;
        }else{
            return 0;
        }
    }
    if(dp[idx][sum] != -1){
        return dp[idx][sum];
    }
    int ans = 0;
    ans += fun(idx + 1, sum + idx) % mod;
    ans %= mod;
    ans += fun(idx + 1, sum) % mod;

    return dp[idx][sum] = ans % mod;
}


signed main(){
    cin >> n;
    memset(dp, -1, sizeof dp);
    const int inv2 = 500000004;
    cout << fun(1, 0) * inv2 % mod << '\n';
}
```

Knapsack Retrival

```
#include <bits/stdc++.h>

using namespace std;
int w[100], v[100], n;
int dp[101][1000001], nxt[101][1000001];

int goriberDP(int in, int rw)
{
    if(in == n)
    {
        return 0;
    }
    if(dp[in][rw] != -1)
    {
        return dp[in][rw];
    }
    int ans = 0;
    int op1 = goriberDP(in + 1, rw);
    if(op1 > ans){
        ans = op1;
        nxt[in][rw] = 0;
    }
    if(w[in] <= rw)
    {
        int op2 = v[in] + goriberDP(in + 1, rw -
w[in]);
        if(op2 > ans){
            ans = op2;
            nxt[in][rw] = 1;
        }
    }
    return dp[in][rw] = ans;
}


signed main(){
    int W;
    cin >> n >> W;
    for(int i = 0; i < n; i++)
    {
        cin >> w[i] >> v[i];
    }
    memset(dp, -1, sizeof dp);
    cout << goriberDP(0, W) << endl;
    int i = 0;
    while(W != 0 and i < n){
        if(nxt[i][W]){
            cout << w[i] << ' ';
            W -= w[i];
        }
        i++;
    }
}
```

Knapsack N Item Pick Retrival

```
#include <bits/stdc++.h>

using namespace std;
int w[100], v[100], n;
int dp[101][1000001], nxt[101][1000001];

int goriberDP(int in, int rw)
{
```

Knapsack N Item Pick Retrival (cont.)

```
    if(in == n)
    {
        return 0;
    }
    if(dp[in][rw] != -1)
    {
        return dp[in][rw];
    }
    int ans = 0;
    for(int i = 0; i * w[in] <= rw; i++){
        int op = v[in] * i + goriberDP(in + 1, rw
- w[in] * i);

        if(op > ans){
            ans = op;
            nxt[in][rw] = i;
        }
    }
    return dp[in][rw] = ans;
}


signed main(){
    int W;
    cin >> n >> W;
    for(int i = 0; i < n; i++)
    {
        cin >> w[i] >> v[i];
    }
    memset(dp, -1, sizeof dp);
    cout << goriberDP(0, W) << endl;
    int i = 0;
    while(W != 0 and i < n){
        if(nxt[i][W]){
            cout << i + 1 << ' ' << nxt[i][W]<<
'\n';
            W -= w[i];
        }
        i++;
    }
}
```

Minimum Coins

```
#include <bits/stdc++.h>
#define int long long
#define all(x) x.begin(), x.end()

using namespace std;

int dp[101][1000001];

signed main(){
    int n, x; cin >> n >> x;

    vector<int> coins(n);
    for(int i = 0; i < n; i++) cin >> coins[i];

    const int inf = 1e9;
    vector<int> dp(x+1, inf);
    dp[0] = 0;

    for(int c : coins){
        for(int j = c; j <= x; j++){
            dp[j] = min(dp[j], dp[j - c] + 1);
```

Minimum Coins (cont.)

```
        }
    }

    if(dp[x] >= inf) cout << -1 << endl;
    else cout << dp[x] << endl;
}
```

Maximum Palindrome Subsequence

```
#include <bits/stdc++.h>
using namespace std;

int dp[101][101];
string s;

int isPal(int i, int j) {
    if (i > j)
        return 0;
    if(i == j)return 1;
    if (dp[i][j] != -1)
        return dp[i][j];
    int ans = 0;
    if(s[i] == s[j]){
        ans = max(ans, 2 + isPal(i + 1, j - 1));
    }
    ans = max(ans, isPal(i + 1, j));
    ans = max(ans, isPal(i, j - 1));

    return dp[i][j] = ans;
}

int main() {
    memset(dp, -1, sizeof(dp));
    int n;
    cin >> n >> s;

    cout << isPal(0, n - 1) << endl;
}
```

Palindrome Check L to R Substring

```
#include <bits/stdc++.h>
using namespace std;

int dp[101][101];
string s;

bool isPal(int i, int j) {
    if (i >= j)
        return true;

    if (dp[i][j] != -1)
        return dp[i][j];

    if (s[i] != s[j])
        return dp[i][j] = false;

    return dp[i][j] = isPal(i + 1, j - 1);
}

int main() {
```

Palindrome Check L to R Substring (cont.)

```
    memset(dp, -1, sizeof(dp));

    int q;
    cin >> q >> s;

    while (q--) {
        int l, r;
        cin >> l >> r;
        l--;
        r--;

        cout << (isPal(l, r) ? "YES" : "NO") <<
'\n';
    }
}
```

Distinct Ordered Number of Ways Make N

```
#include <bits/stdc++.h>
#define int long long
#define all(x) x.begin(), x.end()

using namespace std;

int dp[101][1000001];

signed main()
{
    int n, x;
    cin >> n >> x;

    vector<int> coins(n);
    for (int i = 0; i < n; i++)
        cin >> coins[i];

    const int inf = 1e9;
    vector<int> dp(x + 1, 0);
    dp[0] = 1;
    int mod = 1e9 + 7;
    for (int c : coins)
    {
        for (int j = 1; j <= x; j++)
        {
            if (j - c >= 0)
            {
                dp[j] = (dp[j] + dp[j - c]) % mod;
            }
        }
    }

    if (dp[x] >= inf)
        cout << -1 << endl;
    else
        cout << dp[x] << endl;
}
```

Number of Distinct Ways

```
// https://cses.fi/problemset/task/1635/

#include <bits/stdc++.h>
#define int long long
```

Number of Distinct Ways (cont.)

```
using namespace std;

signed main()
{
    int n, x;
    cin >> n >> x;

    int coins[n + 1];
    for (int i = 1; i <= n; i++)
        cin >> coins[i];

    const int MOD = 1e9 + 7;
    vector<int> dp(x + 1, 0);
    dp[0] = 1;

    for (int i = 1; i <= x; i++)
    {
        for (int j = 1; j <= n; j++)
        {
            if (i - coins[j] >= 0)
            {
                dp[i] = (dp[i] + dp[i - coins[j]])
% MOD;
            }
        }
    }

    cout << dp[x] << endl;
}
```

DATA STRUCTURES

Segment tree

O(log n)

Range sum, point update

```
#include <bits/stdc++.h>
using namespace std;

#define int long long

const int N = 2e5 + 5;

int a[N];
int tree[4 * N];

void build(int node, int l, int r){

    if(l == r){
        tree[node] = a[l];
        return;
    }

    int mid = (l + r) / 2;

    build(node * 2, l, mid);
    build(node * 2 + 1, mid + 1, r);

    tree[node] = tree[node * 2] + tree[node * 2 +
1];
}

int query(int node, int l, int r, int ql, int qr){

    // No overlap
```

Segment tree (cont.) O(log n)

```
    if(r < ql || l > qr)
        return 0;

    // Complete overlap
    if(ql <= l && r <= qr)
        return tree[node];

    // Partial overlap
    int mid = (l + r) / 2;

    return query(node * 2, l, mid, ql, qr) +
           query(node * 2 + 1, mid + 1, r, ql,
qr);
}

void update(int node, int l, int r, int idx, int
val){

    if(l == r){
        tree[node] = val;
        return;
    }

    int mid = (l + r) / 2;

    if(idx <= mid)
        update(node * 2, l, mid, idx, val);
    else
        update(node * 2 + 1, mid + 1, r, idx,
val);

    tree[node] = tree[node * 2] + tree[node * 2 +
1];
}

signed main(){

    int n;
    cin >> n;

    for(int i = 1; i <= n; i++)
        cin >> a[i];

    build(1, 1, n);

    int q;
    cin >> q;

    while(q--){

        int type;
        cin >> type;

        if(type == 1){

            int idx, val;
            cin >> idx >> val;

            update(1, 1, n, idx, val);
        }

        else{

            int l, r;
            cin >> l >> r;

            cout << query(1, 1, n, l, r) << '\n';
        }
    }
```

Segment tree (cont.) O(log n)

```
}
```

DSU O(a(n))

Union by size, path compression

```
#include <bits/stdc++.h>
using namespace std;

#define int long long

const int N = 2e5 + 5;

int parent[N];
int sz[N];

void make(int v){

    parent[v] = v;
    sz[v] = 1;
}

int find(int v){

    if(parent[v] == v)
        return v;

    return parent[v] = find(parent[v]);    // Path
Compression
}

void Union(int a, int b){

    a = find(a);
    b = find(b);

    if(a != b){

        // Union by Size
        if(sz[a] < sz[b])
            swap(a, b);

        parent[b] = a;
        sz[a] += sz[b];
    }
}

signed main(){

    int n, q;
    cin >> n >> q;

    for(int i = 1; i <= n; i++)
        make(i);

    while(q--){

        int type;
        cin >> type;

        if(type == 1){

            int u, v;
            cin >> u >> v;
```

DSU (cont.)

O(a(n))

```
            Union(u, v);
        }

        else{

            int u, v;
            cin >> u >> v;

            if(find(u) == find(v))
                cout << "YES\n";
            else
                cout << "NO\n";
        }
    }
}
```

MISC

2D prefix sum

O(N * M)

```
vector<vector<long long>> pref;
void buildPrefix2D(int n, int m){
    pref.assign(n + 1, vector<long long>(m + 1,
0));
    for(int i = 1; i <= n; i++){
        for(int j = 1; j <= m; j++){

            long long x;
            cin >> x;

            pref[i][j] = x
                       + pref[i-1][j]
                       + pref[i][j-1]
                       - pref[i-1][j-1];
        }
    }
}
long long queryPrefix2D(int r1, int c1, int r2,
int c2){

    return pref[r2][c2]
         - pref[r1-1][c2]
         - pref[r2][c1-1]
         + pref[r1-1][c1-1];
}
```

Diagonal Traversal (Top-Left → Bottom-Right)

```
vector<int> diagonalTraversal(vector<vector<int>>
&a){

    int n = a.size();
    int m = a[0].size();

    vector<int> ans;

    for(int d = 0; d <= n + m - 2; d++){

        int row = min(d, n - 1);
        int col = d - row;

        while(row >= 0 && col < m){
            ans.push_back(a[row][col]);
```

Diagonal Traversal (Top-Left → Bottom-Right) (cont.)

```
            row--;
            col++;
        }
    }

    return ans;
}
```

Diagonal Traversal (Bottom-Left → Top-Right)

```
vector<int> diagonalTraversal(vector<vector<int>>
&a){

    int n = a.size();
    int m = a[0].size();

    vector<int> ans;

    // Start from first column (bottom to top)
    for(int row = n - 1; row >= 0; row--){

        int i = row;
        int j = 0;

        while(i < n && j < m){
            ans.push_back(a[i][j]);
            i++;
            j++;
        }
    }

    // Start from top row (left to right),
skipping (0,0)
    for(int col = 1; col < m; col++){

        int i = 0;
        int j = col;

        while(i < n && j < m){
            ans.push_back(a[i][j]);
            i++;
            j++;
        }
    }

    return ans;
}
```

Inversion Count

O(n log n)

```
long long mergeAndCount(vector<int>& arr, int
left, int mid, int right) {
    vector<int> temp;
    int i = left, j = mid + 1;
    long long inv_count = 0;
    while (i <= mid && j <= right) {
        if (arr[i] <= arr[j]) {
            temp.push_back(arr[i++]);
        } else {
            temp.push_back(arr[j++]);
            inv_count += (mid - i + 1);
        }
    }
```

Inversion Count (cont.)

O(n log n)

```
    while (i ⩽ mid) temp.push_back(arr[i++]);
    while (j ⩽ right) temp.push_back(arr[j++]);
    for (int k = left; k ⩽ right; ++k) {
        arr[k] = temp[k - left];
    }
    return inv_count;
}

long long mergeSortAndCount(vector<int>& arr, int
left, int right) {
    if (left ⩾ right) return 0;
    int mid = (left + right) / 2;
    long long inv_count = 0;
    inv_count += mergeSortAndCount(arr, left,
mid);
    inv_count += mergeSortAndCount(arr, mid + 1,
right);
    inv_count += mergeAndCount(arr, left, mid,
right);

    return inv_count;
}
```